

# Express.js + MongoDB CRUD Tutorial (Part 2 – READ Operation)

---

This tutorial explains how to **read data from MongoDB using Express.js and Mongoose**, and display that data in **EJS (HTML templates)**.

---

## 1. Overview of the CRUD Project Structure

The goal is to:

- Connect Express.js with MongoDB
  - Fetch data from MongoDB
  - Display all records on a home page
  - Display a **single record in detail** using an ID
- 

## 2. Required Package: Mongoose

To work with MongoDB in Express.js, a special package is required:

Install Mongoose

```
npm install mongoose
```

Mongoose acts as a bridge between **Express.js** and **MongoDB**, providing:

- Schema definitions
  - Built-in CRUD methods
  - Validation and type safety
- 

## 3. Steps to Use MongoDB with Express.js

Whenever MongoDB is used with Express.js, these **four steps are mandatory**:

1. Install Mongoose
2. Include (require) Mongoose
3. Connect to MongoDB
4. Define a Model (Collection + Schema)

Once these steps are completed, CRUD operations can be performed.

---

## 4. Including Mongoose in the Project

In the main file (e.g., `index.js`):

```
const mongoose = require("mongoose");
```

---

## 5. Connecting Express.js to MongoDB

### MongoDB Connection String

MongoDB runs locally on:

```
mongodb://localhost:27017
```

Add a database name at the end:

```
mongodb://localhost:27017/contact_crud
```

### Connection Code

```
mongoose.connect("mongodb://localhost:27017/contact_crud")
  .then(() => {
    console.log("Database connected successfully");
  });
```

#### Important Notes:

- If the database does not exist, MongoDB creates it automatically.
- Without a connection, no MongoDB operation can run.

---

## 6. Organizing Database Logic (Models Folder)

To keep the project clean:

- Create a folder named `models`
- All database-related logic goes inside this folder

### MVC Concept (Brief)

- **Model** → Database logic
- **View** → EJS / HTML files
- **Controller** → Routes & logic

---

## 7. Creating a Mongoose Schema

File: `models/contactModel.js`

```
const mongoose = require("mongoose");

const contactSchema = new mongoose.Schema({
  first_name: { type: String },
  last_name: { type: String },
  email: { type: String },
  phone: { type: String },
  address: { type: String }
});
```

### Why Schema is Important:

- Defines allowed fields
  - Enforces data types
  - Prevents invalid data insertion
- 

## 8. Creating a Model (Collection)

```
const Contact = mongoose.model("Contact", contactSchema);
module.exports = Contact;
```

- MongoDB collection is created automatically
  - Collection name becomes `contacts`
- 

## 9. Importing the Model in `index.js`

```
const Contact = require("../models/contactModel");
```

---

## 10. CRUD Operations Overview (MongoDB + Mongoose)

### Create

- `insertOne`
- `insertMany`
- `create` (Mongoose shortcut)

### Read

- `find`
- `findOne`

- `findById`

## Update

- `updateOne`
- `updateMany`
- `findByIdAndUpdate`

## Delete

- `deleteOne`
- `deleteMany`
- `findByIdAndDelete`

---

## 11. Inserting Dummy Data (MongoDB Shell)

```
db.contacts.insertOne({
  first_name: "Jesse",
  last_name: "Pinkman",
  email: "jessepinkman69@gmail.com",
  phone: "8552023696",
  address: "Albuquerque"
});
```

Verify using:

```
db.contacts.find();
```

---

## 12. Reading All Records (Home Page)

Route: Home Page

```
app.get("/", async (req, res) => {
  const contacts = await Contact.find();
  res.render("home", { contacts });
});
```

**Why `async/await` is used:**

- Database operations take time
- `await` pauses execution until data is fetched

---

## 13. Displaying Data in EJS (Table)

## Loop Through Data

```
<% contacts.forEach(contact => { %>
<tr>
  <td><%= contact.first_name %></td>
  <td><%= contact.last_name %></td>
  <td><%= contact.email %></td>
  <td><%= contact.phone %></td>
</tr>
<% }) %>
```

- Dummy rows are removed
- Data is rendered dynamically from MongoDB

---

## 14. Passing MongoDB ID for Actions

MongoDB automatically generates:

```
_id
```

This ID is used for:

- View single contact
- Update contact
- Delete contact

Example:

```
<a href="/contact/<%= contact._id %>">View</a>
```

---

## 15. Reading a Single Record (Show Contact Page)

Route with Parameter

```
app.get("/contact/:id", async (req, res) => {
  const contact = await Contact.findById(req.params.id);
  res.render("showContact", { contact });
});
```

**Why `findById` is better:**

- Cleaner syntax

- No object matching needed
  - Optimized for ID searches
- 

## 16. Displaying Single Contact in EJS

```
<p>First Name: <%= contact.first_name %></p>
<p>Last Name: <%= contact.last_name %></p>
<p>Email: <%= contact.email %></p>
<p>Phone: <%= contact.phone %></p>
<p>Address: <%= contact.address %></p>
```

- No loop needed
  - Only one record is shown
- 

## 17. Preparing for Update & Delete

IDs are also passed in:

- Update links
- Delete links

Example:

```
<a href="/update/<%= contact._id %>">Edit</a>
<a href="/delete/<%= contact._id %>">Delete</a>
```

---

## 18. Summary

- Connect Express.js with MongoDB using Mongoose
  - Define schemas and models
  - Read all records from MongoDB
  - Display data in EJS tables
  - Fetch and display a single record using ID
-